Click here to view the PDF version.

```julia
begin
    using Plots
    using PlutoUI
    using PlutoTeachingTools
    using Printf
    using LaTeXStrings
    using HypertextLiteral: @htl, @htl_str
end
```

Table of Contents

Numerical differentiation

First order derivatives

⚠ TODO ⚠

Construction of finite difference formulas

Determination of finite differences coefficients

Using interpolating polynomials

Computing higher-order derivatives

Numerical stability

⚠ TODO ⚠

Numerical differentiation

In the last chapter we discussed numerical techniques for integration based on quadrature formulas. In this chapter we will consider numerical techniques for performing the inverse operation, namely differentiation.

First order derivatives

Given a regular function $f : [a, b] \to \mathbb{R}$ our goal is to approximate numerically its derivative f' in a point $x \in [a, b]$. Similar to the case of numerical integration we will allow ourselves only to perform n pointwise evaluations $f(t_i)$ with $t_i \in [a, b]$ and $i = 1, \ldots, n$ and take linear combinations of these results.

A first idea goes back to the definition of the derivative $f'(x)$ as the limit $h \to 0$ of the slope of secants over an intervall $[x, x + h]$, i.e.

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

A natural idea is to not fully take the limit, i.e. to take a small $h > 0$ and approximate

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

This immediately leads to the **forward finite difference formula**

$$D_h^+ f(x) = \frac{1}{h}\Big(f(x+h) - f(x)\Big). \tag{1}$$

Note that in this expression we interpret $D_h^+ f(x)$ in the same way as $\frac{d}{dx} f(x)$, i.e. as an operation applied to the function f at the point x.

Without any surprise this can indeed be employed to approximate derivatives. Let us consider the function $f(x) = \sin\left(e^{x+1}\right)$ or

```
f (generic function with 1 method)
```

```
f(x) = sin(exp(x + 1))
```

which has analytical derivative $f'(x) = e^{x+1} \cos\left(e^{x+1}\right)$, i.e. $f'(0) = e \cos(e)$ or

```
exact_value = -2.478349732955235
```

```
exact_value = e * cos(e)
```

Simply evaluating (1) with $h = 10^{-4}$ and $x = 0$ yields

```
(-2.47863, 0.000275671)
```

```
let
    x = 0
    h = 1e-4
    D⁺ₕ = 1/h * (f(x + h) - f(x))
    error = abs(exact_value - D⁺ₕ)

    (D⁺ₕ, error)
end
```

which is already a pretty good approximation to $f'(0)$. For a smaller $h = 10^{-6}$ the result is even better

```
(-2.47835, 2.75667e-6)
```

```
let
    x = 0
    h = 1e-6
    D⁺ₕ = 1/h * (f(x + h) - f(x))
    error = abs(exact_value - D⁺ₕ)

    (D⁺ₕ, error)
end
```

This experiment already points to **linear convergence** (Why ?), so let us investigate the convergence behaviour more closely. We consider a Talyor serios of f around x:

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(\xi)h^2 \qquad \text{for } \xi \in (x, x+h)$$

Considering (1) we thus obtain

$$D_h^+ f(x) = \frac{1}{h}\Big(f(x+h) - f(x)\Big) = f'(x) + \frac{1}{2}f''(\xi)h$$

or

$$\left|f'(x) - D_h^+ f(x)\right| \leq \frac{h}{2} \max_{x\in[a,b]} |f''(x)| = \frac{h}{2}\|f''\|_\infty$$

We just proved the linear convergence of D_h^+:

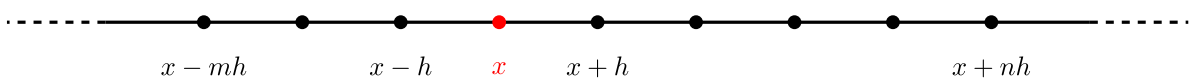
Theorem 1: Convergence of forward finite differences

Given $f : [a, b] \to \mathbb{R}$ a C^2 function, then the forward finite difference formula (1) converges linearly

$$|f'(x) - D_h^+ f(x)| \leq Ch$$

with constant $C = \frac{1}{2}\|f''\|_\infty$.

In building such finite difference formulas we are not restricted to two nodes and evaluations of f. In general one can imagine to approximate the derivative of f at x by taking n points to the right of x, i.e. $x + h, x + 2h, \ldots, x + nh$ and m points to its left, $x - mh, \ldots, x - 2h, x - h$:



A general definition is:

Definition: Finite differences formula

A **finite differences formula** for the $n + m + 1$ equally equispaced **nodes** $x + ih$, $i = -m, \ldots, -1, 0, 1, \ldots, n$ around $x \in [a, b]$ is an expression of the form

$$D_h f(x) = \frac{1}{h} \sum_{i=-m}^{n} w_i f(x + ih), \tag{2}$$

where the **coefficients** $w_{-m}, \ldots, w_0, \ldots w_n$ do not depend on h.

Such a formula is called **consistent** if for a regular function $f : [a, b] \to \mathbb{R}$ it approximates its first derivative for $h \to 0$, i.e.

$$f'(x) = \lim_{h \to 0} D_h f(x).$$

The forward differences formula (1) only employed the two nodes x and $x + h$, i.e. $m = 0$ and $n = 1$. Two other formulas with only two nodes are frequently employed, namely:

- **Backward finite differences:** Instead of constructing the line through x and $x + h$, one can also construct the line through $x - h$ and x, which leads to

$$D_h^- f(x) = \frac{1}{h}\Big(f(x) - f(x-h)\Big) \tag{3}$$

i.e. a two-point formula with $m = -1$ and $n = 0$.

- **Central finite differences:**

$$D_h^c f(x) = \frac{1}{2h}\Big(f(x+h) - f(x-h)\Big) \tag{4}$$

One can think of this formula as the symmetrised form of the forward and backward formulas (1) and (3), obtained by adding half of (1) to half of (3).

⚠ TODO ⚠

Some image explaining these formulas using the slope of lines between points would be good.

```
TODO("Some image explaining these formulas using the slope of lines between points would be good.")
```

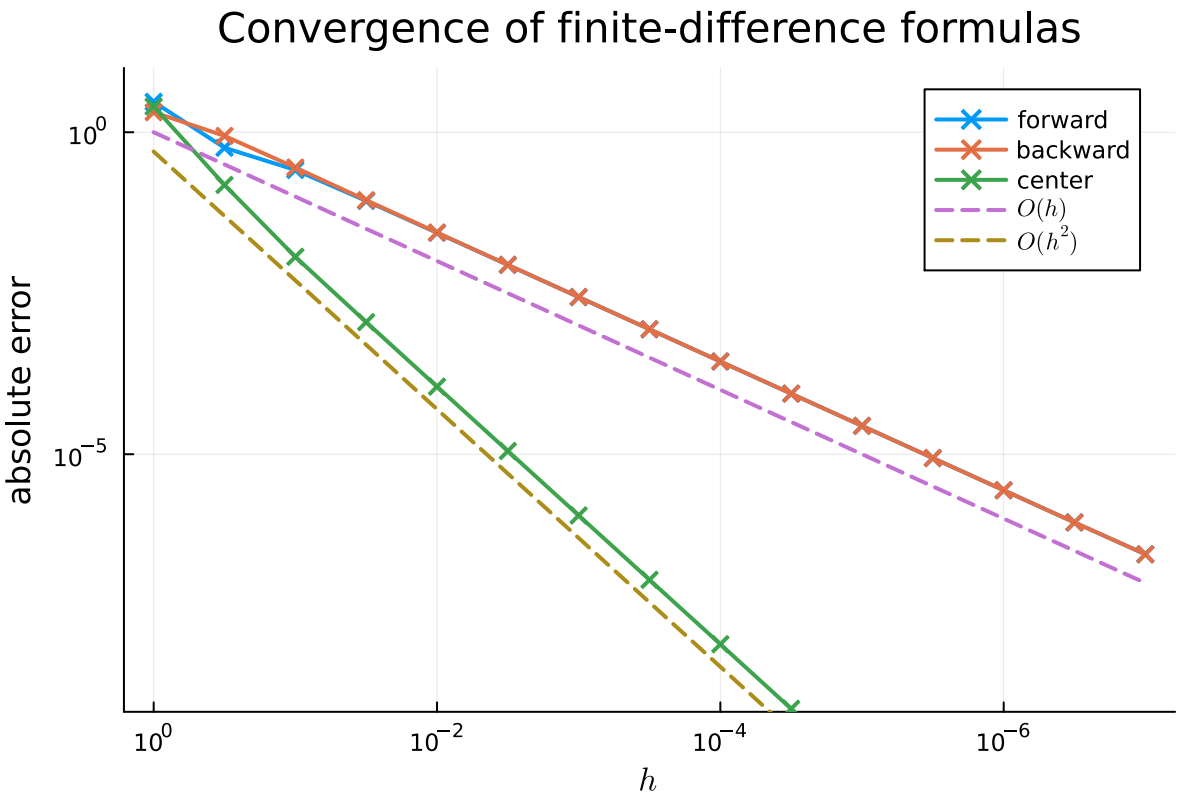
Let us consider the convergence of these three variants for computing the derivative of $f(x) = e^{\sin(x)}$ at $x = 0$. First we construct the range of h values:

```
hs = [10^e for e in 0:-0.5:-7];  # Range of h parameter
```

Then we compute the finite-difference approximations of f' at x:

```
begin
    x = 0
    deriv_forward  = [1/h    * (f(x+h) - f(x))   for h in hs]
    deriv_backward = [1/h    * (f(x)   - f(x-h)) for h in hs]
    deriv_center   = [1/(2h) * (f(x+h) - f(x-h)) for h in hs]
end;
```

Next we compute the errors agains. the reference $f'(x) = 1$ and plot the convergence in a log-log plot:




```julia
let
    p = plot(; yaxis=:log, xaxis=:log, xflip=true, ylims=(1e-9, 10),
           title="Convergence of finite-difference formulas", xlabel=L"h",
           ylabel="absolute error")

    error_forward  = abs.(deriv_forward  .- exact_value)
    error_backward = abs.(deriv_backward .- exact_value)
    error_center   = abs.(deriv_center   .- exact_value)

    plot!(p, hs, error_forward;  label="forward",  lw=2, mark=:x)
    plot!(p, hs, error_backward; label="backward", lw=2, mark=:x)
    plot!(p, hs, error_center;   label="center",   lw=2, mark=:x)

    # Lines for perfect 1st and 2nd order convergence.
    plot!(p, hs, hs;       ls=:dash, label=L"O(h)",   lw=2)
    plot!(p, hs, 0.5hs.^2; ls=:dash, label=L"O(h^2)", lw=2)
end
```

We observe that both forward and backward finite differences converge at about the same rate. Indeed, one easily proves that the **backward finite differences** formula is also of **first order**.

However, the center finite differences formula converges much faster. We again analyse using Taylor's formula:

$$
\begin{aligned}
D_h^c f(x) &= \frac{1}{2h}(f(x+h) - f(x-h)) \\
&= \frac{1}{2h}\Bigg[f(x) + hf'(x) + \frac{h^2}{2}f''(x) + \frac{h^3}{6}f'''(\xi_1) \\
&\qquad - \left(f(x) - hf'(x) + \frac{h^2}{2}f''(x) - \frac{h^3}{6}f'''(\xi_2)\right)\Bigg] \\
&= \frac{1}{2h}\left[2\,hf'(x) + \frac{2h^3}{6}\left(f'''(\xi_1) + f'''(\xi_2)\right)\right] \\
&= f'(x) + \frac{h^2}{6}\left(f'''(\xi_1) + f'''(\xi_2)\right) \\
&\leq f'(x) + \frac{2h^2}{6}\max_{x\in[a,b]}|f'''(x)|
\end{aligned}
$$

where $\xi_1 \in (x, x+h)$ and $\xi_2 \in (x-h, x)$. Therefore central finite differences is of second order:

Theorem 2: Convergence of central finite differences

Given $f : [a,b] \to \mathbb{R}$ a C^3 function, then the central finite difference formula (3) converges **quadratically**

$$|f'(x) - D_h^c f(x)| \leq Ch^2$$

with constant $C = \frac{1}{6}\|f'''\|_\infty = \frac{1}{6}\max_{x\in[a,b]}|f'''(x)|$.

In light of this discussion let us formalise the definition of convergence order for the context of finite differences formulas:

Definition: Convergence order of finite differences

A finite differences formula $D_h f$ of the form (2) with equally spaced nodes of separation h is of **order** p if a constant C indepentent of h (but possibly dependent on f) exists, such that

$$|f'(x) - D_h f(x)| \leq Ch^p$$

as long as the function f is sufficiently regular.

Construction of finite difference formulas

Above we introduced three finite difference formulas, which have been motivated directly from the definition of of the derivative itself and all employed only two points. The construction of more involved formulas is feasible and in fact frequently employed to obtain an even higher convergence order.

We will not discuss all details in this lecture and only sketch two ideas how other finite difference formulas can be obtained in practice. For further discussion and a table of common finite difference formulas see for example chapter 5.4 Driscoll, Brown: Fundamentals of Numerical Computation.

Determination of finite differences coefficients

The definition (2) of a finite difference formula already introduced the general expression

$$\tilde{D}_h f(x) = \frac{1}{h} \sum_{i=-m}^{n} w_i f(x+ih).$$

The integers m and n determine the limits of the sum and thus the number of nodes at which one needs to evaluate the function.

Since function evaluation is usually the expensive step, the values for m and n are usually set based on practical limitations (such as the available computational time or the structure of the computational problem). The key unknown is thus to determine the coefficients w_i, which can be obtained by matching $\tilde{D}_h f(x)$ as closely as possible to $f'(x)$ using Taylor expansions of f. We consider an example:

Example: m=n=1

We consider the case $m = n = 1$, i.e. we want to derive the finite-differences formula using the three nodal points $x-h$, x and $x+h$. For this setting the general formula (2) becomes

$$\tilde{D}_h f(x) = \frac{1}{h}[w_{-1} f(x-h) + w_0 f(x) + w_1 f(x+h)]. \tag{5}$$

Expanding $f(x-h)$ and $f(x+h)$ using Taylor series we have

$$
\begin{aligned}
\tilde{D}_h f(x) = \frac{w_{-1}}{h}\left[f(x) - f'(x)h + \frac{f''(x)}{2}h^2 + O(h^3)\right] \\
+ \frac{w_0}{h} f(x) \\
+ \frac{w_1}{h}\left[f(x) + f'(x)h + \frac{f''(x)}{2}h^2 + O(h^3)\right].
\end{aligned}
$$

Collecting coefficients we obtain

$$
\tilde{D}_h f(x) = \underbrace{\frac{w_{-1} + w_0 + w_1}{h}}_{=0} f(x) + \underbrace{(w_1 - w_{-1})}_{=1} f'(x) + \underbrace{\frac{h\,(w_{-1} + w_1)}{2}}_{=0} f''(x) + O(h^2)
$$

which we want to be as close as possible to $f'(x)$. On the coefficients this imposes the conditions

$$
\begin{cases} w_{-1} + w_0 + w_1 = 0 \\ -w_{-1} \qquad + w_1 = 1 \\ w_{-1} \qquad + w_1 = 0 \end{cases} \quad \Leftrightarrow \quad \underbrace{\begin{pmatrix} 1 & 1 & 1 \\ -1 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix}}_{=\mathbf{A}} \begin{pmatrix} w_{-1} \\ w_0 \\ w_1 \end{pmatrix} = \underbrace{\begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}}_{=\mathbf{b}}
$$

The solution to this system can be easily computed as

```
[-0.5, 0.0, 0.5]
```

```
let
    A = [ 1 1 1;
         -1 0 1;
          1 0 1]
    b = [0;
         1;
         0]
    w = A \ b
end
```

i.e. $w_{-1} = -\frac{1}{2}$, $w_0 = 0$, $w_1 = \frac{1}{2}$, which we insert into (5) to obtain

$$
\tilde{D}_h f(x) = \frac{1}{h}\left(\frac{-1}{2} f(x-h) + \frac{1}{2} f(x+h)\right) = \frac{1}{2h}\Big(f(x+h) - f(x-h)\Big).
$$

We thus recover the central finite differences formula (4) from earlier.

Using interpolating polynomials

Similar to the construction of quadrature formulas, we can also obtain finite difference formulas based on interpolating polynomials.

To rationalise this idea recall the forward finite differences formula (1)

$$D_h^+ f(x) = \frac{1}{h}\Big(f(x+h) - f(x)\Big) = \frac{f(x+h) - f(x)}{h}.$$

We already noted that this equation can be interpreted as the slope of a line going through the points $\big(x, f(x)\big)$ and $\big(x+h, f(x+h)\big)$. Therefore it is equal to the derivative $p_1'(x)$ of a (linear) polynomial interpolating $\big(x, f(x)\big)$ and $\big(x+h, f(x+h)\big)$ as can be easily verified. Indeed, using Lagrange polynomials we easily construct the interpolating polynomial as

$$\begin{aligned} p_1(\xi) &= f(x)\frac{\xi - (x+h)}{x - (x+h)} + f(x+h)\frac{\xi - x}{x+h-x} \\ &= -f(x)\frac{\xi - (x+h)}{h} + f(x+h)\frac{\xi - x}{h} \\ &= \frac{f(x+h) - f(x)}{h}\xi + \frac{xf(x) + hf(x) - xf(x+h)}{h} \end{aligned},$$

which has derivative

$$p_1'(x) = \frac{f(x+h) - f(x)}{h}$$

as required.

This procedure can be generalised: Given the $n+m+1$ nodes $x+ih$ for $i = -m, \dots, n$ of a finite differences formula, we first construct the $(n+m)$-th degree polynomial p_{n+m} interpolating the data $\big((x+ih), f(x+ih)\big)$. Then we build the finite differences formula as

$$D_h f(x) = p_{n+m}'(x).$$

Example n=m=1

We apply the procedure again to the case $m = n = 1$ with the three nodal points $x-h$, x and $x+h$. Using a Lagrange basis we find the second-degree interpolating polynomial p_2 through the points $\big(x-h, f(x-h)\big)$, $\big(x + 0\cdot h, f(x + 0\cdot h)\big)$ and $\big(x+h, f(x+h)\big)$ as

$$
\begin{aligned}
p_2(\xi) = f(x-h)&\frac{(\xi - x)(\xi - (x+h))}{((x-h)-x)((x-h)-(x+h))} \\
&+ f(x)\frac{(\xi-(x-h))(\xi-(x+h))}{(x-(x-h))(x-(x+h))} \\
&+ f(x+h)\frac{(\xi-(x-h))(\xi-x)}{((x+h)-(x-h))((x+h)-x)} \\
= f(x-h)&\frac{(\xi-x)(\xi-x-h)}{2h^2} + f(x)\frac{(\xi-x+h)(\xi-x-h)}{-h^2} \\
&+ f(x+h)\frac{(\xi-x+h)(\xi-x)}{2h^2}.
\end{aligned} \tag{6}
$$

Its derivative is

$$
\begin{aligned}
p_2'(\xi) = f(x-h)&\frac{\xi-x+\xi-x-h}{2h^2} + f(x)\frac{\xi-x+h+\xi-x-h}{-h^2} \\
&+ f(x+h)\frac{\xi-x+h+\xi-x}{2h^2}
\end{aligned}
$$

such that at $\xi = x$ we get the finite-difference formula

$$
p_2'(x) = \frac{-f(x-h)}{2h} + \frac{f(x+h)}{2h},
$$

which coincides again with the central finite difference formula (4).

Computing higher-order derivatives

The methods sketched in the aforementioned section also allow us to build finite-difference formulas to build higher-order derivatives.

For example, based on the interpolated polynomial (6) on the nodal points $x-h$, x and $x+h$, we obtain a formula for **approximating the second derivatives** as

$$
D_h^2 f(x) = p_2''(x) = \frac{f(x-h) - 2f(x) + f(x+h)}{h^2}. \tag{7}
$$

This formula is also of **second order** as can be checked easily using a Taylor series.

Numerical stability

One of the key principles behind all finite difference formulas we considered was that they approximate the derivative, in the sense that $f'(x) = \lim_{h\to 0} D_h\, f(x)$. As a result we would expect results to become more and more accurate as h gets smaller.

Let's check this with the function

```
g (generic function with 1 method)
```

```julia
g(x) = exp(-1.3x)
```

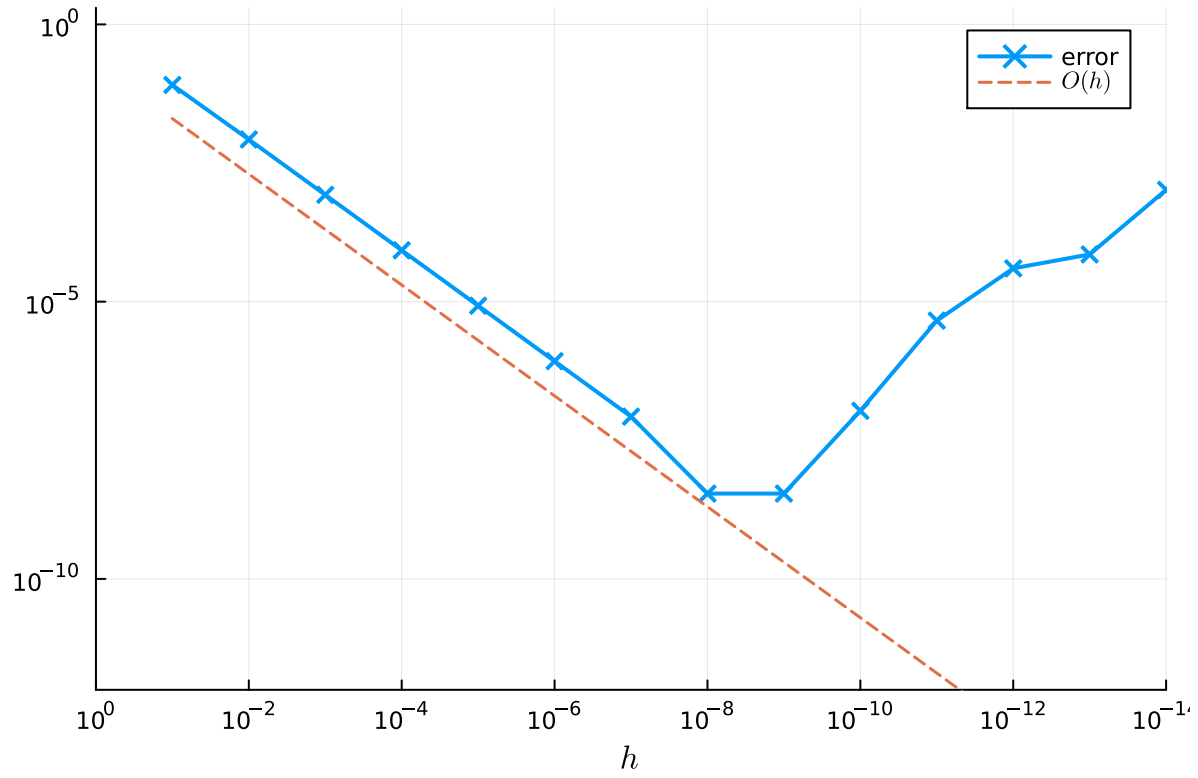
which at $x = 0$ clearly has the derivative $g'(0) = -1.3$. Using the forward finite differences formula we compute

```julia
begin
    X = 0.0
    reference = -1.3  # g'(X) = g'(0)

    all_h = [10^i for i in -1:-1.0:-14]
    all_error = Float64[]  # Error of D⁺ₕ
    for h in all_h
        D⁺ₕ = 1/h * (g(X + h) - g(X))
        error = abs(D⁺ₕ - reference)
        @printf "h=%.1e   D⁺ₕ=%.12f   error=%.6e\n" h D⁺ₕ error
        push!(all_error, error)
    end
end
```

```
h=1.0e-01   D⁺ₕ=-1.219045690794   error=8.095431e-02
h=1.0e-02   D⁺ₕ=-1.291586497971   error=8.413502e-03
h=1.0e-03   D⁺ₕ=-1.299155366048   error=8.446340e-04
h=1.0e-04   D⁺ₕ=-1.299915503662   error=8.449634e-05
h=1.0e-05   D⁺ₕ=-1.299991550041   error=8.449959e-06
h=1.0e-06   D⁺ₕ=-1.299999154991   error=8.450089e-07
h=1.0e-07   D⁺ₕ=-1.299999915494   error=8.450610e-08
h=1.0e-08   D⁺ₕ=-1.299999996540   error=3.459820e-09
h=1.0e-09   D⁺ₕ=-1.299999996540   error=3.459820e-09
h=1.0e-10   D⁺ₕ=-1.300000107562   error=1.075625e-07
h=1.0e-11   D⁺ₕ=-1.300004548455   error=4.548455e-06
h=1.0e-12   D⁺ₕ=-1.299960139534   error=3.986047e-05
h=1.0e-13   D⁺ₕ=-1.300071161836   error=7.116184e-05
h=1.0e-14   D⁺ₕ=-1.298960938811   error=1.039061e-03
```

Plotting this error graphically yields:




```
begin
    plot(all_h, all_error; xaxis=:log, yaxis=:log, mark=:x, lw=2, label="error",
    xflip=true, xlabel=L"h", ylims=(1e-12, 2), xlims=(1e-14, 1))
    plot!(all_h, 0.2all_h, label=L"O(h)", lw=1.5, ls=:dash)
end
```

We notice that the derivative formula gives a good approximation to the exact derivative $g'(0) = -1.3$ for $h = 10^{-8}$. However, if the node distance h is further descreased the approximation deteriorates. This is a result of the round-off error in the finite-precision floating-point arithmetic of the computer. We take another look at the forward finite difference formula

$$D_h^+ f(x) = \frac{f(x+h) - f(x)}{h}.$$

In particular computing the difference $f(x+h) - f(x)$ becomes problematic as h gets smaller, since this implies that the values of $f(x+h)$ and $f(x)$ get more and more similar. As a result the difference $f(x+h) - f(x)$ (and thus the overall numerical derivative $D_h^+ f(x)$) become less and less precise.

Let us quantify this effect more precisely. When computing the function $f(x)$ the employed algorithm will always suffer from a small round-off error[1]. Instead of evaluating $f(x)$ the computer thus actually evaluates

$$\hat{f}(x) = f(x)(1+\eta),$$

where the round-off error η is on the order of 10^{-16}. Note that in standard floating-point formats the round-off error is always a *relative error*. The precise value of η in general depends on x, but we further have $|\eta| \leq \epsilon_M$ where for standard double-precision floating-point numbers ϵ_M is about $2 \cdot 10^{-16}$.

When evaluating the finite-difference formula the computer will thus actually evaluate

$$\begin{aligned}
\widehat{D}_h^+ f(x) &= \frac{\hat{f}(x+h) - \hat{f}(x)}{h} \\
&= \frac{f(x+h)(1+\eta_1) - f(x)(1+\eta_2)}{h} \\
&= \frac{f(x+h) - f(x)}{h} + \frac{\eta_1}{h} f(x+h) - \frac{\eta_2}{h} f(x) \\
&= f'(x) + \frac{f''(\xi)}{2} h + \frac{\eta_1}{h} f(x+h) - \frac{\eta_2}{h} f(x),
\end{aligned}$$

where $|\eta_1| \leq \epsilon_M$, $|\eta_2| \leq \epsilon_M$ and $\xi \in (x, x+h)$. Collecting everything we obtain the error of the computed finite-difference approximation to $f'(x)$ as

$$\begin{aligned}
|f'(x) - \widehat{D}_h^+ f(x)| &\leq \frac{h}{2} \max_{x \in [a,b]} |f''(x)| + 2 \max_{x \in [a,b]} |f(x)| \frac{\epsilon_M}{h} \\
&= \underbrace{\frac{h}{2} \|f''\|_\infty}_{\text{trunc. error}} + \underbrace{\frac{2\epsilon_M}{h} \|f\|_\infty}_{\text{round-off error}}
\end{aligned} \tag{8}$$

We notice there are two error terms in (8). One is proportional to h (finite differences truncation error) and one is proportional to $1/h$ (due to round-off errors). As $h \to 0$ the first term thus derceases as the finite-difference approximation gets better. However, if h is taken too small the second error growing as $1/h$ will dominate and our approximation will be bad.

A natural question is to ask is which h gives the best sweet spot and balances both errors. Utilising equation (8) the total error of the approximation is bounded by

$$\varepsilon(h) = \frac{h}{2} \|f''\|_\infty + \frac{2\epsilon_M}{h} \|f\|_\infty.$$

This function has a single minimum, which we can compute by

$$0 = \frac{d\varepsilon}{dh} \qquad \Rightarrow \qquad \frac{\|f''\|_\infty}{2} - \frac{2\,\epsilon_M\,\|f\|_\infty}{h^2} = 0$$

The optimal node distance h_{opt}, which minimises the error, is thus

$$h_{\text{opt}} = \sqrt{\frac{4\|f\|_\infty}{\|f''\|_\infty}}\sqrt{\epsilon_M}. \tag{9}$$

In our example we have $\|f\|_\infty \approx |f(0)| = 1$ and $\|f''\|_\infty \approx |f''(0)| = 1.69$ and therefore h_{opt} is of order $\sqrt{10^{-16}} = 10^{-8}$, which we also observed numerically.

[1]:

> Note that on top of round-off errors there may be additional error contributions. A typical case is when evaluating $f(\mathbf{x})$ involves itself an iterative procedure, e.g. if $f(\mathbf{x}) = \mathbf{z}$ and $\mathbf{z}$ is the solution to a linear systems $\mathbf{A}\mathbf{z} = \mathbf{x}$ computed using a conjugate gradient algorithm. In this case the iterative procedure is rarely solved perfectly, but stopped once a relative residual norm has been reached, which causes additional error contributions.

⚠ TODO ⚠

Show the theoretical error behaviour in the plot

```
TODO("Show the theoretical error behaviour in the plot")
```

Note that in (8) the h dependence of the first error term (finite difference truncation error) depends on the order of the finite difference formula. For an order p method the error will thus have the form

$$\varepsilon(h) = C_1 h^p + C_2 \epsilon_M h$$

with appropriate constants C_1 and C_2, which leads to an optimal value of order $\sqrt[p+1]{\epsilon_M}$. We summarise:

Observation: Optimal nodal spacing h for finite differences

For computing an approximate derivative with a finite-difference method of order p in the presence of round-off errors in the function evaluations f, the optimal spacing of nodes satisfies roughly

$$h_{\text{opt}} \approx \sqrt[p+1]{\epsilon_M}$$

To illustrate this graphically we continue our example of computing the derivative of $g(x) = e^{-1.3x}$ at $x = 0$, which has the reference result

```
-1.3
```

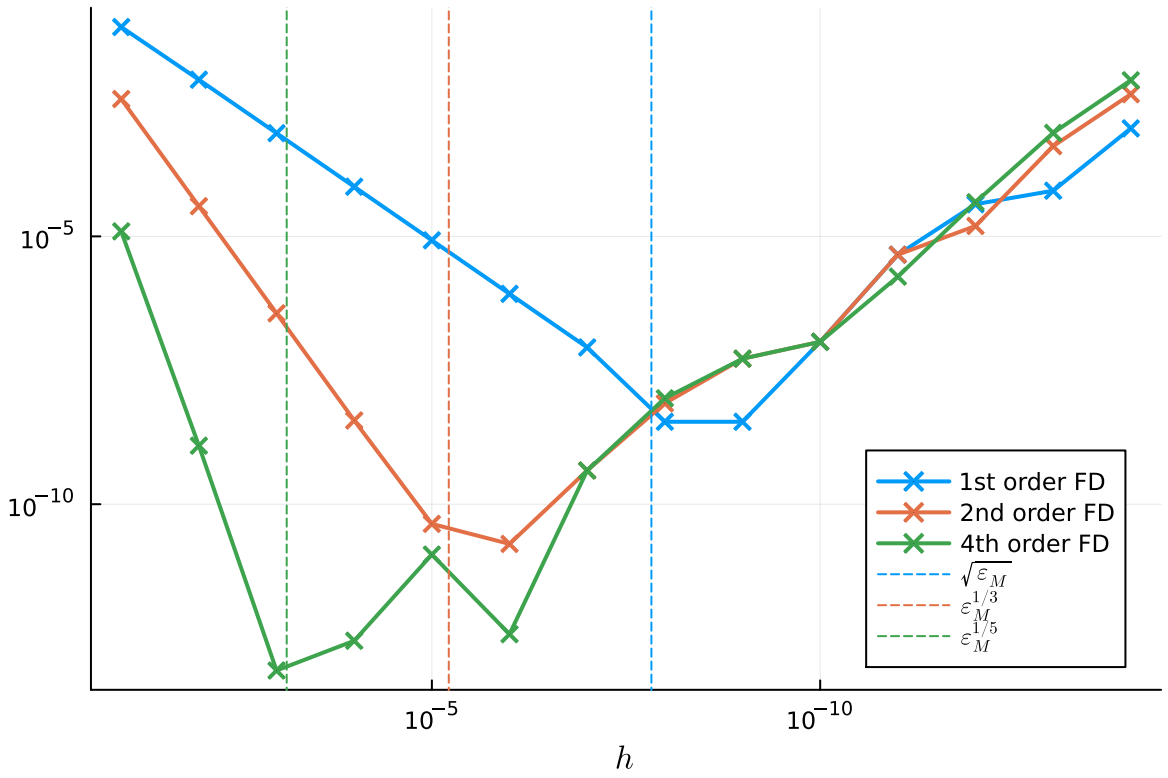
```
reference
```

We consider three finite difference (FD) formulas, namely

- **1st order:** $D_1\, g(x) = D_h^+\, g(x) = \dfrac{g(x+h) - g(x)}{h}$ (forward finite differences)
- **2nd order:** $D_2\, g(x) = D_h^c\, g(x) = \dfrac{g(x+h) - g(x-h)}{2h}$ (central finite differences)

- **4th order:** $D_4\, g(x) = \dfrac{g(x-2h) - 8\, g(x-h) + 8\, g(x+h) - g(x+2h)}{12h}$

We see that as h shrinks the errors are initially dominated by the truncation error, which dicreases most rapidly for the 4th order formula. However, the increasing round-off error eventually dominates the behaviour as the truncation error continues to decrease. As the order increases the crossover point between truncation error and round-off error moves further to the left and further down. Thus higher-order methods are generally more accurate as the numerically problematic small h values can be avoided.




```julia
let
    error_fd1 = Float64[]
    error_fd2 = Float64[]
    error_fd4 = Float64[]

    for h in all_h
        diff_fd1 = 1/h   * (g(X + h) - g(X))
        diff_fd2 = 1/2h  * (g(X + h) - g(X - h))
        diff_fd4 = 1/12h * (g(X -2h) - 8g(X-h) + 8g(X+h) - g(X+2h))

        push!(error_fd1, abs.(diff_fd1 - reference))
        push!(error_fd2, abs.(diff_fd2 - reference))
        push!(error_fd4, abs.(diff_fd4 - reference))
    end

    p = plot(; xaxis=:log, yaxis=:log, mark=:x, lw=2, xflip=true, xlabel=L"h", legend=:bottomright)
    plot!(p, all_h, error_fd1; label="1st order FD", mark=:x, lw=2)
    plot!(p, all_h, error_fd2; label="2nd order FD", mark=:x, lw=2,)
    plot!(p, all_h, error_fd4; label="4th order FD", mark=:x, lw=2)

    vline!(p, [sqrt(eps())], label=L"\sqrt{ε_M}", c=1, ls=:dash)
    vline!(p, [cbrt(eps())], label=L"ε_M^{1/3}",  c=2, ls=:dash)
    vline!(p, [eps()^(1/5)], label=L"ε_M^{1/5}",  c=3, ls=:dash)
end
```

Numerical analysis

1. Introduction
2. The Julia programming language
3. Revision and preliminaries
4. Root finding and fixed-point problems
5. Interpolation
6. Direct methods for linear systems
7. Iterative methods for linear systems
8. Eigenvalue problems
9. Numerical integration
10. Numerical differentiation
11. Initial value problems
12. Boundary value problems